Lab – SQL In-Built Date Functions

Objective

The objective of this lab is to understand and use SQL Server built-in date functions for:

- Getting the current date and time
- Adding or subtracting date parts
- Formatting dates
- Finding differences between dates
- Extracting day, month, and year
- Getting month and weekday names
- Finding the last date of a month
- Calculating age
- Filtering table records based on dates
- Grouping records year-wise and month-wise
- Applying aggregate functions with dates

PART 1: BASIC DATE FUNCTIONS

1. GETDATE()

`GETDATE()` returns the current system date and time.

Syntax

```
GETDATE()
```

Example

```
SELECT GETDATE() AS CURRENT_DATE_TIME;
```

Possible output:

```
2026-07-12 14:30:25.123
```

Using Alias

```
SELECT GETDATE() AS COLUMN_NAME;
```

An alias changes only the displayed column name.

---

2. DATEADD()

`DATEADD()` adds or subtracts a specified amount of time from a date.

Syntax

```
DATEADD(datepart, number, date)
```

Parameters

```
datepart → What do you want to add or subtract?

number   → How much do you want to add or subtract?

date     → Starting date
```

Example: Add Days

```
SELECT DATEADD(DAY, 10, GETDATE());
```

Meaning:

```
Current Date
     ↓
Add 10 Days
     ↓
New Date
```

Example: Add Years

```
SELECT DATEADD(YEAR, 3, GETDATE());
```

Example: Subtract Months

Use a negative number to subtract.

```
SELECT DATEADD(MONTH, -4, GETDATE());
```

Important Rule

```
Positive Number
      ↓
Add Date Part


Negative Number
      ↓
Subtract Date Part
```

COMMON DATEPART VALUES

Date Part	Meaning
`YEAR`	Year
`QUARTER`	Quarter
`MONTH`	Month
`DAY`	Day
`DAYOFYEAR`	Day of year
`WEEK`	Week
`WEEKDAY`	Weekday
`HOUR`	Hour
`MINUTE`	Minute
`SECOND`	Second

Common abbreviations:

Date Part	Abbreviation
`YEAR`	`YY`, `YYYY`
`MONTH`	`MM`, `M`
`DAY`	`DD`, `D`
`HOUR`	`HH`
`MINUTE`	`MI`, `N`
`SECOND`	`SS`, `S`

For lab queries, using complete names such as `YEAR`, `MONTH`, `DAY`, and `HOUR` is easier to understand.

---

3. CONVERT() FOR DATE FORMATTING

SQL Server provides `CONVERT()` styles to display dates in different formats.

Syntax

```
CONVERT(data_type, date_value, style_number)
```

General Pattern

```
SELECT CONVERT(VARCHAR, GETDATE(), style_number);
```

The style number controls the output format.

IMPORTANT DATE FORMATS

Style	Output Pattern
`0` or `100`	mon dd yyyy hh
`1` or `101`	mm/dd/yyyy
`3` or `103`	dd/mm/yyyy
`6` or `106`	dd mon yyyy
`7` or `107`	Mon dd, yyyy
`10` or `110`	mm-dd-yyyy

Style	Output Pattern
`11` or `111`	yyyy/mm/dd
`12` or `112`	yyyymmdd
`23`	yyyy-mm-dd

Example

```
SELECT CONVERT(VARCHAR, GETDATE(), 106);
```

Possible result:

```
12 Jul 2026
```

Another Example

```
SELECT CONVERT(VARCHAR, GETDATE(), 107);
```

Possible result:

```
Jul 12, 2026
```

Important

Remember:

```
CONVERT(VARCHAR, date, style)
```

The third argument is the style number.

---

4. DATEDIFF()

`DATEDIFF()` calculates the difference between two dates.

Syntax

```
DATEDIFF(datepart, start_date, end_date)
```

Example: Difference in Days

```
SELECT DATEDIFF(DAY, '2025-01-01', '2025-01-11');
```

Result:

```
10
```

Example: Difference in Months

```
SELECT DATEDIFF(MONTH, '2025-01-01', '2025-05-01');
```

Result:

```
4
```

Example: Difference in Hours

```
SELECT DATEDIFF(HOUR, start_date, end_date);
```

Execution

```
Start Date

     ↓

End Date

     ↓

Choose Unit

DAY / MONTH / YEAR / HOUR

     ↓

DATEDIFF()

     ↓
```

```
Difference
```

Important Behavior

`DATEDIFF()` counts date-part boundaries crossed.

It does not always calculate complete elapsed units.

For example:

```
SELECT DATEDIFF(YEAR, '2025-12-31', '2026-01-01');
```

returns:

```
1
```

even though only one day has passed.

---

5. DAY()

`DAY()` extracts the day number from a date.

Syntax

```
DAY(date)
```

Example

```
SELECT DAY('2026-08-25');
```

Result:

```
25
```

---

6. MONTH()

`MONTH()` extracts the month number from a date.

Syntax

```
MONTH(date)
```

Example

```
SELECT MONTH('2026-08-25');
```

Result:

```
8
```

---

7. YEAR()

`YEAR()` extracts the year from a date.

Syntax

```
YEAR(date)
```

Example

```
SELECT YEAR('2026-08-25');
```

Result:

```
2026
```

EXTRACTING DAY, MONTH AND YEAR TOGETHER

Pattern

```
SELECT
    DAY(date_value) AS DAY_VALUE,
    MONTH(date_value) AS MONTH_VALUE,
    YEAR(date_value) AS YEAR_VALUE;
```

Execution

```
Date

 ↓

DAY()   → Day Number

MONTH() → Month Number

YEAR()  → Year Number
```

---

8. DATENAME()

`DATENAME()` returns the date part as a character string.

Syntax

```
DATENAME(datepart, date)
```

Example: Month Name

```
SELECT DATENAME(MONTH, GETDATE());
```

Possible result:

```
July
```

Example: Weekday Name

```
SELECT DATENAME(WEEKDAY, GETDATE());
```

Possible result:

```
Sunday
```

---

9. DATEPART()

`DATEPART()` returns the specified part of a date as an integer.

Syntax

```
DATEPART(datepart, date)
```

Example

```
SELECT DATEPART(MONTH, GETDATE());
```

Possible result:

```
7
```

DATENAME() VS DATEPART()

Function	Returns
`DATENAME()`	Character string
`DATEPART()`	Integer

Example:

```
SELECT DATENAME(MONTH, GETDATE());
```

Result:

```
July
```

While:

```
SELECT DATEPART(MONTH, GETDATE());
```

Result:

```
7
```

Easy rule:

```
DATENAME
    ↓
NAME
    ↓
July


DATEPART
    ↓
NUMBER
    ↓
7
```

---

10. EOMONTH()

`EOMONTH()` returns the last date of the month.

Syntax

```
EOMONTH(date)
```

Example

```
SELECT EOMONTH('2026-02-10');
```

Result:

```
2026-02-28
```

Current Month Pattern

```
EOMONTH(GETDATE())
```

Add or Subtract Months

`EOMONTH()` can also accept an optional month offset.

Syntax

```
EOMONTH(date, month_offset)
```

Example:

```
SELECT EOMONTH(GETDATE(), 1);
```

returns the last date of the next month.

CALCULATING AGE

Age calculation commonly uses `DATEDIFF()`.

Basic Age in Years

Pattern

```
DATEDIFF(YEAR, birth_date, GETDATE())
```

Important

This basic pattern counts year boundaries.

Therefore, it may return an age that is one year too high when the birthday has not occurred yet in the current year.

More Accurate Completed Years Pattern

A more accurate calculation can check whether the birthday has occurred this year.

General Idea

```
Calculate Year Difference

        ↓

Find Birthday Date in Current Year

        ↓

Has Birthday Occurred?

       /

     YES  NO

      ↓    ↓

   Keep   Subtract 1
   Age
```

For basic lab questions, follow the method taught by your faculty.

Age in Years and Remaining Months

The main logic is:

```
Birth Date

     ↓

Current Date

     ↓

Calculate Total Months

     ↓
```

```
Years  = Total Months / 12

Months = Total Months % 12
```

General Pattern

```
DATEDIFF(MONTH, birth_date, GETDATE()) / 12
```

for years, and:

```
DATEDIFF(MONTH, birth_date, GETDATE()) % 12
```

for remaining months.

Remember that `DATEDIFF(MONTH, ...)` counts month boundaries, so a fully precise age calculation requires additional adjustment.